

FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

Análise e Gestão de horários III

Relatório Final do Projeto Integrador

**Isabel Oliveira
Mário Silva
Martim Pernafor**



Licenciatura em Engenharia Informática e Computação

Professor Orientador: Prof. Daniel Castro Silva

16 de julho de 2025

Resumo

Este projeto teve como principal objetivo a evolução de uma plataforma de gestão de horários desenvolvida por estudantes de edições anteriores do Projeto Integrador. A plataforma permite importar os horários tal como estão no momento no SIGARRA, proporcionando uma visualização mais eficiente e compacta das aulas. Além disso, permite para editar manualmente os horários e apresenta, em linguagem natural, uma lista das alterações efetuadas. Concentrámo-nos na implementação de novas funcionalidades e na melhoria de outras já existentes, com vista a facilitar o processo de edição manual de horários e a integração futura com um sistema de otimização automática.

Os nossos esforços resultaram em melhorias significativas nos seguintes aspetos:

- **Seleção de Aulas em Paralelo:** por motivos de restrições logísticas, algumas aulas tem de ser agrupadas em grupos, que denominamos de grupos de aulas em paralelo. Como é impossível agrupar automaticamente, foi criada uma interface para agrupar as aulas manualmente.
- **Nova visualização por Unidade Curricular (UC):** foi implementada uma interface dedicada à visualização e edição de horários por UC, facilitando as alterações feitas em aulas de UC's específicas.
- **Deteção de conflitos:** foi criado um módulo dedicado à identificação de incompatibilidades entre aulas (por sala, turma ou docente), permitindo maior fiabilidade na deteção de conflitos.
- **Exportação de alterações:** foi desenvolvido um sistema completo de registo e exportação das alterações feitas nos horários, com deteção automática de conflitos e ordenação das alterações com base nas dependências existentes.

Estas melhorias têm como objetivo agilizar o processo de construção e revisão de horários, reduzindo erros e aumentando a eficiência das comissões responsáveis por essa tarefa.

Índice

1	Introdução	1
1.1	Breve resumo do enquadramento e motivação	1
1.2	Objetivos e resultados	1
1.3	Estrutura do relatório	1
2	Descrição do Problema	3
2.1	Requisitos	3
2.2	Metodologia utilizada	3
2.3	Intervenientes, papéis e responsabilidades	4
2.4	Atividades desenvolvidas	4
3	Desenvolvimento da solução	6
3.1	Conceção e Implementação	6
3.1.1	Seleção das Aulas em Paralelo	6
3.1.2	Visualização e Edição de Horário por UC	7
3.1.3	Exportação de Alterações	9
3.1.4	Deteção de Conflitos	11
3.2	Solução desenvolvida	12
3.2.1	Visualização e Edição de Horário por UC	12
3.2.2	Exportação de Alterações	13
3.2.3	Deteção de conflitos	13
3.3	Validação	14
4	Conclusões	15
4.1	Resultados alcançados	15
4.2	Lições Aprendidas	15
4.3	Trabalho futuro	16
	Referências	17

Capítulo 1

Introdução

A geração de horários, que respeitem restrições logísticas e que sejam eficientes, é um desafio complexo e comum em muitas instituições, entre as quais se destacam as de ensino. É um processo que envolve múltiplas restrições complexas como a disponibilidade de salas e docentes, distribuição equilibrada da carga horária, entre outras.

1.1 Breve resumo do enquadramento e motivação

No caso da FEUP, este processo de geração de horários eficientes passa por gerar uma solução de horários que respeite as numerosas restrições logísticas e depois otimizá-la manualmente para melhorar a sua qualidade. Para esse processo de otimização, os responsáveis pelos horários da LEIC tem usado *spreadsheets* de Excel como ferramenta para visualizar os horários e aplicar alterações. Com o objetivo de facilitar e automatizar este processo, edições anteriores do Projeto Integrador [1][2] desenvolveram e melhoraram uma plataforma desenhada para visualizar horários e efetuar trocas manuais de forma mais eficiente e intuitiva. Trocas manuais serão sempre inevitáveis e, desta forma, os responsáveis pela gestão dos horários da LEIC terão a possibilidade de usar uma ferramenta desenhada para esse propósito.

Paralelamente, em 2024, foi também criado por Miguel Lopes - *alumni* do MIEIC - um otimizador de horários, no contexto da sua dissertação do M.EIC [3], permitindo combinar a eficiência da otimização automática com a flexibilidade da edição manual. Nesta edição do Projeto Integrador, o trabalho incidiu sobre a evolução da plataforma existente, com foco na reformulação e adição de funcionalidades, correção de bugs, melhorias na usabilidade e integração do otimizador com o restante sistema.

1.2 Objetivos e resultados

Este projeto consiste em melhorar uma plataforma que permite a visualização e alteração de horários, previamente criada e desenvolvida por dois grupos de alunos da LEIC, mas que estava inacabada. Com a aprimoração desta plataforma espera-se que a mesma se torne uma ferramenta útil na criação e gestão de horários e que permita agilizar e facilitar este processo. Para isto, tivemos como objetivo a criação de uma página para selecionar grupos de aulas em paralelo, a criação de uma nova vista de horários, reformulação da exportação em linguagem natural e a integração da plataforma com o otimizador de Miguel Lopes. Todos os objetivos foram cumpridos, exceto a integração com o otimizador.

1.3 Estrutura do relatório

Para além desta introdução, o relatório está organizado em três secções principais:

1. **Introdução** - Nesta secção apresentam-se um breve resumo do enquadramento e da motivação, os objetivos e resultados e esta descrição da estrutura do relatório.
2. **Descrição do problema** – Esta secção expõe mais detalhadamente o problema da geração de horários como um todo, o trabalho que já tinha sido desenvolvido até o momento e os requisitos para este projeto.
3. **Solução Desenvolvida** – Nesta parte é descrito o processo de construção da solução proposta, abordando as funcionalidades num nível mais abstrato e também a implementação delas num nível mais técnico.
4. **Conclusão** – Esta última secção reúne os principais resultados alcançados, refletindo sobre os objetivos do projeto, e propõe possíveis direções para trabalhos futuros que possam complementar e aprofundar o trabalho realizado.

Capítulo 2

Descrição do Problema

No caso da FEUP, o processo de geração de horários, sem conflitos logísticos e eficientes, passa por gerar uma solução de horários que respeite as restrições logísticas e depois alterar de forma a otimizá-la para melhorar a sua qualidade. A cuja plataforma em que fizemos melhorias serve principalmente para visualizar os horários e poder aplicar alterações, visualizar as alterações aplicadas, e exportar a lista delas em linguagem natural. Como o processo para realizar trocas no SIGARRA é feito através de um email a explicar as trocas que se pretendem fazer, sentiu-se a necessidade de criar esta plataforma de forma a visualizar o resultado das trocas e obter a lista delas, em linguagem natural, já prontas para serem enviadas ao SIGARRA.

2.1 Requisitos

Os principais requisitos do trabalho foram:

- **Seleção das aulas em paralelo** - Apresentação das aulas que possivelmente são em paralelo para o utilizador poder seleccionar quais delas realmente o são .
- **Nova visualização por Unidade Curricular (UC)** - Criação de uma vista específica que organiza a informação horária por disciplina, que proporciona uma visualização simplificada em comparação com o modo de visualização já existente.;
- **Funcionalidade de exportação de alterações** - Sistema para exportar o histórico de modificações realizadas no horário, de modo legível e organizado, permitindo documentar e analisar as mudanças efetuadas. Em adição, as alterações devem ser reordenadas de modo a reduzir ao máximo possível o numero de conflitos que dependem da ordem;
- **Integração do otimizador de horários** - Integração na plataforma de um módulo de otimização automática capaz de gerar horários eficientes considerando múltiplos parâmetros.

2.2 Metodologia utilizada

O desenvolvimento do trabalho seguiu uma abordagem Ágil [4], muito semelhante ao Scrum [5] com *sprints* e reuniões de acompanhamento semanais com o Prof. Daniel Castro Silva. Nessas reuniões era apresentado o progresso desenvolvido desde a reunião anterior, recebia-se *feedback* e discutia-se eventuais alterações a aplicar e as tarefas a realizar até a reunião seguinte.

Para o desenvolvimento do projeto, usámos o GitHub ¹ para facilitar a colaboração em grupo na escrita do código. A elaboração do relatório foi feita na plataforma online Overleaf ², dedicada à edição de documentos em formato LaTeX [6], que permite a escrita em simultâneo de múltiplos utilizadores.

Pontualmente, quando algum dos elementos do grupo não podia estar presencialmente na FEUP para as reuniões semanais, participavam remotamente através do Discord ³ ou Zoom ⁴.

2.3 Intervenientes, papéis e responsabilidades

O Prof. Daniel Castro Silva desempenhou um papel semelhante ao de *Product Owner*, conforme definido na framework Scrum - esteve envolvido na definição de funcionalidades e prioridades, bem como na validação e fornecimento de *feedback* das funcionalidades desenvolvidas. Para além do papel de *Product Owner*, o Prof. Daniel Castro Silva assumiu também as responsabilidades típicas de um *Scrum Master*, nomeadamente na orientação do projeto, acompanhamento do progresso e no esclarecimento de dúvidas relacionadas com obstáculos no desenvolvimento.

As responsabilidades de cada membro do grupo foram divididas da seguinte forma:

- Isabel Oliveira: Ficou encarregue da seleção das aulas em paralelo e dos *popups* aquando das trocas que envolvam aulas em paralelo ou em simultâneo. Ficou também encarregue, juntamente com o Mário, da nova vista por Unidades Curriculares.
- Mário Silva: Ficou encarregue, com a Isabel, da nova vista por Unidades Curriculares, tendo começado a trabalhar nela antes da Isabel.
- Martim Pernafort: Ficou encarregue das funcionalidades de exportação das alterações realizadas e deteção de conflitos entre aulas.

2.4 Atividades desenvolvidas

Em baixo temos um diagrama de Gantt que mostra a distribuição temporal das atividades desenvolvidas ao longo do projeto. Este diagrama permite visualizar de forma clara e organizada as tarefas realizadas, os seus períodos de execução e a sobreposição entre elas, facilitando o acompanhamento do progresso e o planeamento global da equipa.

¹Ver mais em github.com (consultado em fevereiro de 2025)

²Ver mais em overleaf.com (consultado em junho de 2025)

³Ver mais em discord.com (consultado em fevereiro de 2025)

⁴Ver mais em zoom.us (consultado em abril de 2025)

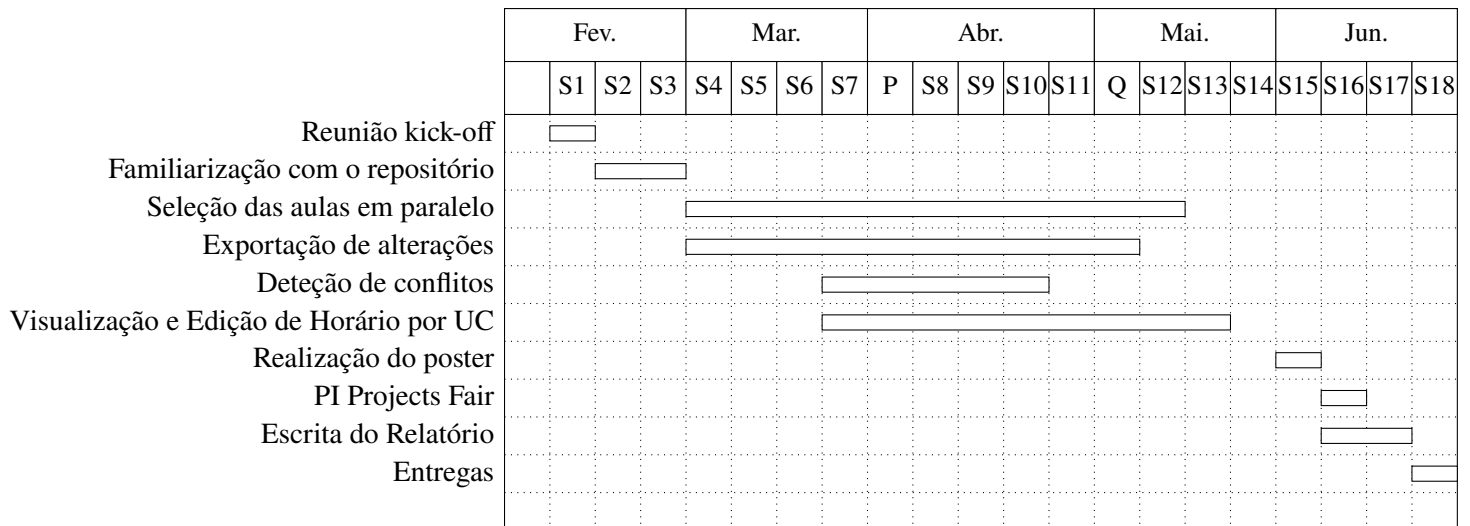


Figura 2.1: Diagrama de Gantt.

Capítulo 3

Desenvolvimento da solução

Neste capítulo é descrito o trabalho desenvolvido para alcançar os resultados esperados.

Se for o caso de um protótipo de software, são apresentados os requisitos, a arquitetura da solução, o desenvolvimento e a validação do protótipo.

Requisito emergente:

- **Detetor de conflitos** - Embora não previsto inicialmente, tornou-se necessário desenvolver um módulo para identificação de incompatibilidades nos horários, garantindo assim a qualidade das soluções geradas.

3.1 Conceção e Implementação

3.1.1 Seleção das Aulas em Paralelo

Grupos de aulas em paralelo são grupos de aulas práticas da mesma Unidade Curricular (e do mesmo curso) que deverão ser leccionadas ao mesmo tempo. São aulas dadas em separado, em salas diferentes e professores diferentes, mas por motivos que não nos cabem, a FEUP exige que sejam dadas ao mesmo tempo. Isso implica que qualquer mudança no horário de uma aula tenha de se refletir nas restantes aulas do grupo. Como são sempre dadas ao mesmo tempo, nos horários elas estão sempre lado a lado - daí o seu nome. Não existe forma de identificar esses grupos automaticamente no parsing dos horários, portanto é necessário que a comissão responsável pelos horários dos cursos os especifique. Apesar de ser impossível de identificar automaticamente esses grupos quando obtemos os horários iniciais, sabemos que as aulas nesse horário, respeita todas as restrições existentes - o que implica que as aulas dos grupos de aulas em paralelo estão a ser leccionadas ao mesmo tempo. Assim, se extraírmos todos os grupos de aulas práticas da mesma UC que estejam a ser dadas ao mesmo tempo, temos uma lista de grupos que poderão ser ser grupos de aulas em paralelo. Para distinguir, nesses grupos, as aulas que fazem parte de um grupo de aulas em paralelo ou as aulas que por acaso são dadas ao mesmo tempo, criámos uma página que mostra as aulas que estão a ser leccionadas ao mesmo tempo e que permite ao utilizador seleccionar quais dessas aulas são aulas em paralelo.

Cada bloco de aulas a ser dado ao mesmo tempo está agrupado num bloco amarelo. Para cada grupo de aulas a ser dado ao mesmo tempo é possível escolher vários grupos separados de aulas em paralelo. Cada *checklist* representa um grupo de aulas em paralelo e o utilizador selecciona as aulas para cada grupo. Assim que selecciona uma aula num grupo, a mesma aula nos restantes grupos é desativada - a mesma aula não pode estar seleccionada em dois grupos diferentes. Claro que se o utilizador deseleccionar, elas passam a estar disponíveis. Para tornar a seleção mais rápida adicionou-se um botão "Selecionar Todas".

Esta página aparece automaticamente quando um projeto novo é criado, mas o utilizador pode acedê-la

Selecionar Aulas em Paralelo

L.EA L.EC L.EEC L.EGI L.EIC L.EM L.EMAT L.EF M.EEC M.EGI

M.EIC M.EM MESW

Curso M.EIC

► Arquitetura de Sistemas de Software M.EIC010
► Desenho e Desenvolvimento de Jogos Digitais M.EIC013
▼ Laboratório de Gestão de Projetos M.EIC006

Sexta, 16:00

Aula com as turmas: 1MEIC01
Aula com as turmas: 1MEIC02
Aula com as turmas: 1MEIC03
Aula com as turmas: 1MEIC04
Aula com as turmas: 1MEIC05
Aula com as turmas: 1MEIC06
Aula com as turmas: 1MEIC07

☐ Selecionar todas
☒ 1MEIC01
☒ 1MEIC02
☐ 1MEIC03
☐ 1MEIC04
☐ 1MEIC05
☐ 1MEIC06
☐ 1MEIC07

☐ Selecionar todas
☐ 1MEIC01
☐ 1MEIC02
☒ 1MEIC03
☒ 1MEIC04
☐ 1MEIC05
☐ 1MEIC06
☐ 1MEIC07

☐ Selecionar todas
☐ 1MEIC01
☐ 1MEIC02
☐ 1MEIC03
☐ 1MEIC04
☐ 1MEIC05
☐ 1MEIC06
☐ 1MEIC07

Curso M.EM

▼ Introdução ao Projeto de Máquinas M.EM009

Quinta, 18:00

Aula com as turmas: 1MEM-IPM04

☐ Selecionar todas
☐ 1MEM-IPM04

Guardar Seleção

Cancelar

Figura 3.1: Página de Seleção das aulas em paralelo

facilmente para editar a qualquer momento.

3.1.2 Visualização e Edição de Horário por UC

Contextualização e Estratégia de Implementação

A funcionalidade de visualização e edição de horários por Unidade Curricular (UC) foi criada como uma alternativa mais simples e robusta à vista principal do horário, a qual apresentava problemas de formatação e inconsistência nas alterações. Esta nova abordagem concentra-se exclusivamente numa UC de cada vez, utilizando uma grelha dinâmica baseada em dias da semana e intervalos de 30 minutos.

Componentes da Implementação

A implementação envolve uma divisão clara entre front-end e back-end, com os seguintes componentes principais:

- **Template `uc_view.html`:**
 - Grelha com colunas de Segunda a Sábado e linhas de 30 minutos (8:00–19:30)
 - Cada aula é um bloco `.aula-box` com atributos `data-*` que indicam posição e identificação
 - Aulas sobrepostas são desenhadas lado a lado com larguras proporcionais
- **JavaScript (`uc_view.js`):**
 - Gestão da interação por clique (seleção de aula, docente, célula)
 - Permite movimentar aulas para células vazias
 - Permite trocar aulas entre si ou docentes entre aulas
 - Envia pedidos AJAX ao back-end e atualiza o DOM com os resultados
- **Back-end (`views.py`):**
 - `uc_view`: gera o HTML da grelha de horário para a UC
 - `uc_changes`: processa alterações de tipo `move`, `swap` ou `teacher`
 - `swap_teachers` e `swap_aulas`: endpoints especializados para trocas entre aulas e docentes

Fluxo de Interação do Utilizador

1. Renderização Inicial

- É desenhada uma grelha com todas as aulas da UC selecionada
- Cada aula é representada por um bloco div com informações de sala, docentes e tipo

2. Seleção e Ações

- O utilizador clica para selecionar uma aula, docente ou célula
- O segundo clique define a ação:
 - Aula + Célula vazia: mover aula
 - Aula + Aula: trocar horários
 - Docente + Docente: trocar entre aulas

3. Validação

- Envia-se o pedido via AJAX para o back-end
- A operação é validada (verifica-se existência de conflitos)

4. Atualização Visual

- Em caso de sucesso, o DOM é atualizado dinamicamente
- Caso haja erro, o bloco é revertido e apresentada uma mensagem

Principais Dificuldades Técnicas

• Mapeamento Temporal da Grelha

- A estrutura antiga centrada em turmas era incompatível com esta vista
- Solução: uso de data-dia e data-hora em cada célula para facilitar indexação e manipulação

• Gestão de Sobreposições

- Aulas simultâneas são desenhadas com larguras ajustadas proporcionalmente
- Implementação de lógica no template para divisão correta do espaço horizontal

• Sincronização Front-End/Back-End

- A interface é responsiva a alterações
- O estado visual é atualizado apenas após confirmação do servidor

Resultados Obtidos

- Visualização clara e direta do horário de uma UC
- Interações rápidas para edição e reorganização
- Detecção automática de conflitos (docente, sala, turma)
- Troca visual e lógica de docentes entre aulas
- Maior robustez, reduzida complexidade e maior confiança por parte dos utilizadores

Tecnologias utilizadas

Justificação da Utilização de jQuery Mesmo sabendo dos problemas que a utilização de bibliotecas poderão causar no futuro, por se tornarem obsoletas e dificultarem o desenvolvimento a longo prazo — a utilização de jQuery neste projeto foi uma decisão ponderada e justificada pelos seguintes motivos:

- **Compatibilidade com código existente:** Esta biblioteca já era utilizada em várias partes do código da interface. Introduzir uma abordagem diferente apenas para esta funcionalidade poderia gerar inconsistências e duplicação de esforço.
- **Rapidez de desenvolvimento:** A simplicidade da API do jQuery permitiu implementar interações com o DOM e chamadas assíncronas de forma rápida e segura.
- **Isolamento controlado:** O uso de jQuery foi restrito a componentes específicos e não afeta a base da aplicação. Caso se opte futuramente por eliminar a dependência da biblioteca, a substituição será localizada e de baixo impacto.
- **Equilíbrio entre pragmatismo e boas práticas:** Reconhecendo os riscos associados ao uso prolongado de bibliotecas, esta decisão foi tomada como uma solução pragmática a curto/médio prazo, privilegiando a estabilidade e integridade do sistema num contexto com prazos limitados e recursos fixos.

Assim, o uso de jQuery não foi uma escolha arbitrária, mas sim uma decisão técnica informada, com preocupações de sustentabilidade e manutenibilidade claramente identificadas e mitigadas.

3.1.3 Exportação de Alterações

Contexto e Arquitetura

A funcionalidade de Exportação de Alterações tem como propósito agrupar e reordenar um conjunto de alterações realizadas nos horários, de modo a facilitar a sua inserção manual na plataforma responsável pela gestão de horários. Para tal, a ordenação das alterações segue dois princípios fundamentais:

- **Agrupamento por Unidades Curriculares (UCs)**, garantindo uma organização lógica das modificações;
- **Reordenação interna por UC**, minimizando conflitos dependentes da ordem de aplicação.

A arquitetura da solução baseia-se em quatro componentes essenciais:

- **Modelo de Aula:** Armazena os metadados completos de cada aula (horário, sala, turmas e docentes), servindo como base para comparação entre estados anterior e posterior às alterações.
- **Gestor de Alterações:** Representa cada modificação como um par de estados (original e modificado), permitindo rastrear a evolução individual de aulas.
- **Grafo de Dependências:** Estrutura que relaciona alterações dentro de uma mesma UC, identificando:
 - Conflitos potenciais entre modificações;
 - Relações de precedência para resolução automática.
- **Coordenador Central:** Orquestra a execução do processo, desde a agregação inicial por UC até à geração da lista final ordenada de alterações, garantindo consistência global.

Fluxo de Processamento de Alterações

1. Comparação entre Bases de Dados

- Realiza uma comparação entre o estado inicial e atual da base de dados
- Identifica as aulas modificadas e cria uma instância para representar a versão inicial e atual de cada uma das aulas em questão
- As instâncias da aula em ambos os estados representa uma alteração, todas as alterações, à medida que são detetadas, são guardadas numa lista desordenada
- Ao finalizar a comparação, as alterações detetadas são guardadas num ficheiro json que se torna invalido caso seja realizada uma nova alteração dentro da plataforma

2. Processamento de Alterações

A etapa de comparação das bases de dados tem como resultado uma lista que contem todas as alterações realizadas. No entanto, estas alterações, não só estão desordenadas, como também, não contêm informação relativamente aos conflitos existentes. Surge assim a necessidade de processar esta lista de modo a obter os dados em falta.

- As alterações da lista são passadas, uma de cada vez, para a classe que atua como Coordenador Central
- Ao serem adicionadas são identificadas tanto as suas dependências (conflitos com o estado inicial do horário, potencialmente serão resolvidos), como os seus conflitos (conflitos reais, não resolvíveis por reordenação)
- Cada alteração é adicionada ao grafo de dependências responsável por gerir a UC na qual a aula alterada se enquadra (caso ainda não exista um grafo para a UC em questão é criado nesta fase)

Nota importante! As alterações que contêm conflitos (reais) são adicionadas diretamente a uma lista referente a dependências não resolvidas de modo a não interferir com o funcionamento correto do algoritmo.

3. Gestão de Conflitos

Após a ultima etapa, passamos de ter (conceptualmente) uma lista desordenada de alterações realizadas para uma lista de alterações devidamente agrupada por UC e com as suas dependências e conflitos identificados.

Nesta fase as alterações vão passar pelo processo de reordenação. Esta reordenação é realizada relativamente a cada UC (representada por um grafo) e, sendo, é essencial compreender como é que este grafo funciona.

- O grafo, como explicado previamente, representa todas as alterações relativas a uma determinada UC
- Os nós representam uma alteração realizada com a adição de uma lista de aulas que identificam as dependências associadas à alteração em questão e uma lista de aulas que identificam os seus conflitos. Caso um nó contenha dependências é categorizado como conflituoso
- As arestas são usadas para representar as dependências entre as alterações, ou seja, servem para conectar os nós conflituosos com os nós que representam alterações em aulas contidas na lista de dependências do nó conflituoso em questão

• Construção do Grafo

- Para cada nó conflituoso:
 - (a) Identifica outros nós responsáveis por realizar alterações nas aulas que inicialmente causam uma dependência no nó que está a ser analisado.
 - (b) Cria arestas para os nós que foram encontrados

(c) Classifica arestas como:

- * **Solução:** Se o nó destino não tem conflitos
- * **Restrição:** Caso o nó destino também seja conflituoso

- **Ordenação**

- **Processo Principal:**

- * Algoritmo recursivo que:
 - (a) Identifica nós sem conflitos
 - (b) Adiciona-os à lista ordenada final
 - (c) Remove suas arestas de saída
 - (d) Repete até esgotar todas as dependências resolvíveis

3.1.4 Detecção de Conflitos

Premissas do Sistema

- **Consistência inicial:** A base de dados original é considerada livre de conflitos.
- **Abordagem completa:** Todas as alterações realizadas no horário são consideradas na análise comparativa.

Algoritmo de Detecção

O processo de deteção de conflitos faz uso do mecanismo de exportação de modo a identificar quais aulas são responsáveis por causar conflitos entre si.

A principal diferença entre ambos é que para a deteção de conflitos não há necessidade de efetuar comparações com o estado inicial da base de dados fazendo assim com que não seja necessária a existencial de uma lista de conflitos reais e de dependências (apenas necessitaria a de conflitos reais)

Funcionamento

- Os dados são adaptados de modo a extrair os ids de todas as aulas com conflitos
- Essa lista é passada para uma classe que atua como gestor
- São criados dicionários dentro da classe ao agrupar os conflitos encontrados por docente, sala ou turma
- Nestes dicionários as chaves servem como identificador do docente/sala/turma tendo como respetivo conteúdo uma lista de alterações que envolvem a chave em questão (e.g. Para um professor procura na lista de conflitos todas as aulas que o envolvam e, caso ainda não esteja na lista, é adicionado)

Resultados

Para cada docente/sala/turma que contenha aulas com conflitos é criada uma entrada no respetivo dicionário com:

- Identificação do docente/sala/turma como chave
- Lista ordenada dos conflitos que envolvem o mesmo

3.2 Solução desenvolvida

3.2.1 Visualização e Edição de Horário por UC

A plataforma passou a disponibilizar uma nova interface dedicada à visualização detalhada do horário por Unidade Curricular (UC), com o objetivo de facilitar a análise isolada de cada UC e permitir um maior controlo sobre as alterações e o seu impacto.

Navegação Inicial A vista principal de horários foi expandida com a adição de uma nova funcionalidade que permite ao utilizador selecionar diretamente uma Unidade Curricular (UC) para filtrar o horário apresentado.

Após selecionar o curso e o ano, o utilizador pode utilizar o novo menu suspenso “UC” para escolher uma Unidade Curricular específica, como ilustrado na Figura 3.2.

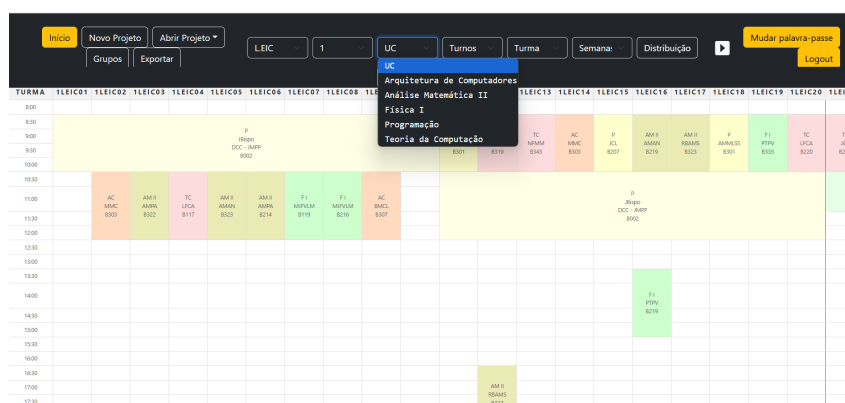


Figura 3.2: Menu de seleção de Unidade Curricular na vista principal

Visualização Detalhada por UC Ao selecionar uma UC, o sistema expande a visualização, exibindo todas as aulas associadas. Cada aula é representada com os seguintes dados:

- Horário (dia da semana, hora de início e fim)
- Sala atribuída
- Docente(s) responsáveis
- Turma(s) associada(s)

HORÁRIO	SEGUNDA	TERÇA	QUARTA	QUINTA	SEXTA	SÁBADO
8:00						
8:30	AM II R24MS B323	AM II AMAN B219	AM II CMC B335	AM II AMPA B123		
9:00				AM II R24MS B323	AM II PCA B335	AM II IARL B335
9:30						
10:00						
10:30						
11:00	AM II AMPA B214	AM II AMAN B323	AM II CMC B335	AM II AMPA B219		AM II IARL B331
11:30						
12:00						
12:30						
13:00						
13:30						
14:00						
14:30						
15:00				AM II CBPF B340	AM II JAFOC B003	
15:30						
16:00						
16:30						
17:00	AM II R24MS B322		AM II R24MS B216	AM II JAFOC B003		
17:30						
18:00						
18:30						
19:00						
19:30						

Figura 3.3: Vista detalhada de uma UC(Análise Matemática II)

3.2.2 Exportação de Alterações

A plataforma disponibiliza uma interface web para exportação das alterações de horário, desenvolvida com o objetivo de apresentar a informação de forma clara e sistemática. A interface organiza as alterações por Unidade Curricular (UC), apresentando inicialmente uma lista compacta de todas as UCs que contêm pelo menos uma aula com modificações. Esta abordagem permite ao utilizador navegar eficientemente através das diversas alterações.



Figura 3.4: Lista de UC's

Quando o utilizador seleciona uma UC específica, a interface expande a visualização para apresentar todas as alterações associadas a essa UC. Cada alteração é identificada por um número único sequencial e pelo respetivo intervalo horário (dia, hora inicial - hora final), facilitando a rápida localização da aula em questão. A identificação exata da aula é feita através das turmas associadas, um critério escolhido por ser mais fiável do que outros identificadores, especialmente em situações onde múltiplas aulas da mesma UC ocorrem em horários sobrepostos.

As alterações são categorizadas em dois grupos distintos: alterações não conflituosas e alterações potencialmente conflituosas. No primeiro grupo, incluem-se todas as modificações que não geram conflitos de recursos (salas, docentes ou turmas). O segundo grupo compreende alterações que apresentam conflitos. Caso se tratem de conflitos que serão resolvidos no decorrer das trocas, a interface apresenta explicitamente a relação entre a alteração conflitua e a(s) alteração(ões) que a resolve(m), criando assim um mapa completo de dependências. Esta situação ocorre somente quando o conflito em questão dependa de uma alteração realizada dentro de outra UC.

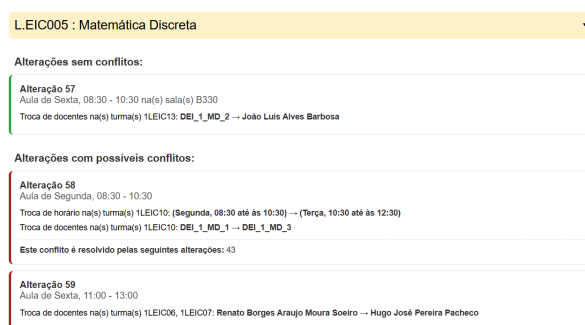


Figura 3.5: Lista de alterações

A implementação desta interface seguiu princípios de design centrados no utilizador, garantindo que mesmo utilizadores não técnicos possam compreender facilmente as alterações propostas.

3.2.3 Deteção de conflitos

De forma semelhante à exportação, a plataforma disponibiliza uma página na qual são apresentados os conflitos detetados.

Diferente da exportação que é destinada a ser usada apenas quando o processo de alterações já se encontra concluído, a página de conflitos é suposto servir como suporte para quem está responsável por construir o horário.

Nesta página podemos averiguar os conflitos gerados pelas alterações realizadas pelo utilizador de modo a conseguir detetar, caso existam conflitos, quais aulas são responsáveis pelo conflito existente.

Conflito entre:	
L.EIC001 - Álgebra Linear e Geometria Analítica (1065)	
Sexta, 10:30 - 12:00	
Turmas: 1LEIC04, 1LEIC05	
Docentes: 240312	
Salas: B342	
L.EIC002 - Análise Matemática I (1098)	
Sexta, 11:00 - 13:00	
Turmas: 1LEIC03	
Docentes: 641825	
Salas: B342	
L.EIC005 - Matemática Discreta (1089)	
Sexta, 09:00 - 11:00	
Turmas: 1LEIC01, 1LEIC02, 1LEIC03, 1LEIC04, 1LEIC05, 1LEIC06, 1LEIC07, 1LEIC08, 1LEIC09	
Docentes: 565525, 652136	
Salas: B002	

Figura 3.6: Lista de alterações

3.3 Validação

A validação da solução desenvolvida foi realizada com base em revisões frequentes por parte do professor orientador, que acompanhou o projeto ao longo de todas as fases. Em reuniões semanais, foram apresentados os progressos, discutidas as funcionalidades implementadas e recolhido feedback detalhado, o qual permitiu identificar problemas, corrigir falhas e ajustar prioridades de desenvolvimento. Esta validação contínua assegurou que a plataforma fosse evoluindo de acordo com os requisitos definidos inicialmente e com as necessidades reais da gestão de horários na FEUP.

Capítulo 4

Conclusões

Neste capítulo são sumariados os resultados alcançados e as lições aprendidas. Finalmente, são apresentadas as limitações do trabalho e são propostas melhorias e trabalho futuro.

4.1 Resultados alcançados

O principal objetivo desta iteração do projeto integrador consistiu na implementação de funcionalidades essenciais para a plataforma de gestão de horários em desenvolvimento.

Tarefas Concluídas

- **Exportação de Alterações:** Sistema completo para registo e exportação de alterações realizadas no horário
- **Deteção de Conflitos:** Módulo funcional para identificação de incompatibilidades horárias
- **Tratamento de Aulas Paralelas e Simultâneas:** Mecanismos para gestão de aulas paralelo e em simultâneo
- **Vista por Unidade Curricular:**
 - Visualização correta por UC implementada
 - Funcionalidade de alteração entre aulas disponível

Tarefas Não Iniciadas

- **Integração com o Otimizador:** Prevista para as próximas iterações

4.2 Lições Aprendidas

Ao longo do desenvolvimento deste projeto, foram várias as aprendizagens importantes, tanto a nível técnico como organizacional. Refletimos de seguida sobre os principais pontos a destacar:

(a) **Evolução Técnica e Aprendizagem de Novas Ferramentas**

Este projeto representou um forte desafio técnico, obrigando-nos a explorar tecnologias com as quais ainda não tínhamos trabalhado, como por exemplo o desenvolvimento de uma API com Django. Para além disso, aprofundámos os nossos conhecimentos em áreas já conhecidas, reforçando particularmente as nossas capacidades de desenvolvimento web.

(b) **Importância do Feedback Contínuo**

O feedback regular por parte do tutor revelou-se essencial para o progresso do projeto. Permitiu esclarecer dúvidas, corrigir rapidamente problemas de conceção ou implementação, e reajustar prioridades consoante o estado do projeto. Esta orientação contínua contribuiu para um progresso mais estruturado e consistente.

4.3 Trabalho futuro

Embora os objetivos principais do projeto tenham sido atingidos, existem ainda várias áreas com potencial de melhoria e desenvolvimento adicional. Destacamos as seguintes direções futuras:

- **Conclusão da implementação da vista por UC's**

Foram já desenvolvidas funcionalidades base para visualização e navegação por Unidade Curricular. No entanto, permanece por concluir a integração de algumas funcionalidades.

- **Integração do otimizador**

Um próximo passo relevante será a integração do algoritmo de otimização desenvolvido por Miguel Lopes (alumni do MEIC).

- **Correção de bugs na vista principal**

Durante o desenvolvimento, foram identificados vários problemas de desformatação na vista principal, nomeadamente após operações de troca de aulas. Estes problemas afetam a consistência visual e a usabilidade da interface.

Referências bibliográficas

- [1] Bárbara Carvalho, José Ramos and Lucas Sousa e Luís Cabral. *Análise e Gestão de Horários I*. Relatório Técnico Relatório 1. Faculdade de Engenharia da Universidade do Porto, 2023.
- [2] Alexandre Correia et al. *Análise e Gestão de Horários I*. Relatório Técnico Relatório 2. Faculdade de Engenharia da Universidade do Porto, 2024.
- [3] Miguel Lopes. «Otimização de Horários Académicos com Algoritmos Genéticos». Tese de Mestrado. Faculdade de Engenharia da Universidade do Porto, 2023.
- [4] FCT NOVA ExecEd. *Em que consiste a metodologia Agile?* <https://execed.fct.unl.pt/consiste-a-metodologia-agile/>. [Online; acedido 15 de junho de 2025]. 2025.
- [5] Scrum.org. *What is Scrum?* <https://www.scrum.org/resources/what-scrum-module>. [Online; acedido 15 de junho de 2025]. 2025.
- [6] LaTeX Project. *LaTeX – A Document Preparation System*. <https://www.latex-project.org/about/>. [Online; acedido 15 de junho de 2025]. 2025.

]